ParkPing Zone Predictor

1. Import libraries & read the CSV

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("./data/IIoT_Smart_Parking_Management.csv")
print(df.shape)
df.head()
```

```
(1000, 28)
```

Out[1]:

	Timestamp	Parking_Spot_ID	Sensor_Reading_Proximity	Sensor_Reading_Pressure
0	2021-01-01 00:00:00.000000000	20	1.023651	1.541461
1	2021-01-02 06:39:16.756756756	49	3.903349	1.621719
2	2021-01-03 13:18:33.513513513	38	10.315709	6.292374
3	2021-01-04 19:57:50.270270270	31	6.588039	1.659870
4	2021-01-06 02:37:07.027027027	8	8.213969	3.278467

5 rows × 28 columns

2. Keep only what we need

In [2]:
```
df = df[["Timestamp", "Parking_Spot_ID", "Parking_Lot_Section", "Occupancy_Status"]]
df.head()
```

Out[2]:

	Timestamp	Parking_Spot_ID	Parking_Lot_Section	Occupancy_Status
0	2021-01-01 00:00:00.000000000	20	Zone A	Occupied
1	2021-01-02 06:39:16.756756756	49	Zone A	Occupied
2	2021-01-03 13:18:33.513513513	38	Zone B	Vacant
3	2021-01-04 19:57:50.270270270	31	Zone B	Vacant
4	2021-01-06 02:37:07.027027027	8	Zone C	Occupied

3. Check if the data is healthy, along with the min & max

In [3]:
```
print("empty cells:", df.isnull().sum().sum())
print("copied rows:", df.duplicated().sum())
print("smallest spot ID:", df["Parking_Spot_ID"].min())
print("biggest spot ID:", df["Parking_Spot_ID"].max())
```

```
empty cells: 0
copied rows: 0
smallest spot ID: 1
biggest spot ID: 50
```

In [4]:
```
print(df["Parking_Lot_Section"].value_counts())
print(df.groupby("Parking_Lot_Section")["Parking_Spot_ID"].nunique()) #Count differ
print(df["Occupancy_Status"].value_counts())
```

```
Parking_Lot_Section
Zone A    382
Zone B    307
Zone C    197
Zone D    114
Name: count, dtype: int64
Parking_Lot_Section
Zone A    50
Zone B    50
Zone C    49
Zone D    44
Name: Parking_Spot_ID, dtype: int64
Occupancy_Status
Occupied    550
Vacant      450
Name: count, dtype: int64
```

4. Make the three clues and the answer

In [5]:
```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
df["Hour"] = df["Timestamp"].dt.hour
df["Day"] = df["Timestamp"].dt.day_name()
df["Occupied"] = df["Occupancy_Status"].map({"Occupied": 1, "Vacant": 0}) #represen
df.head()
```

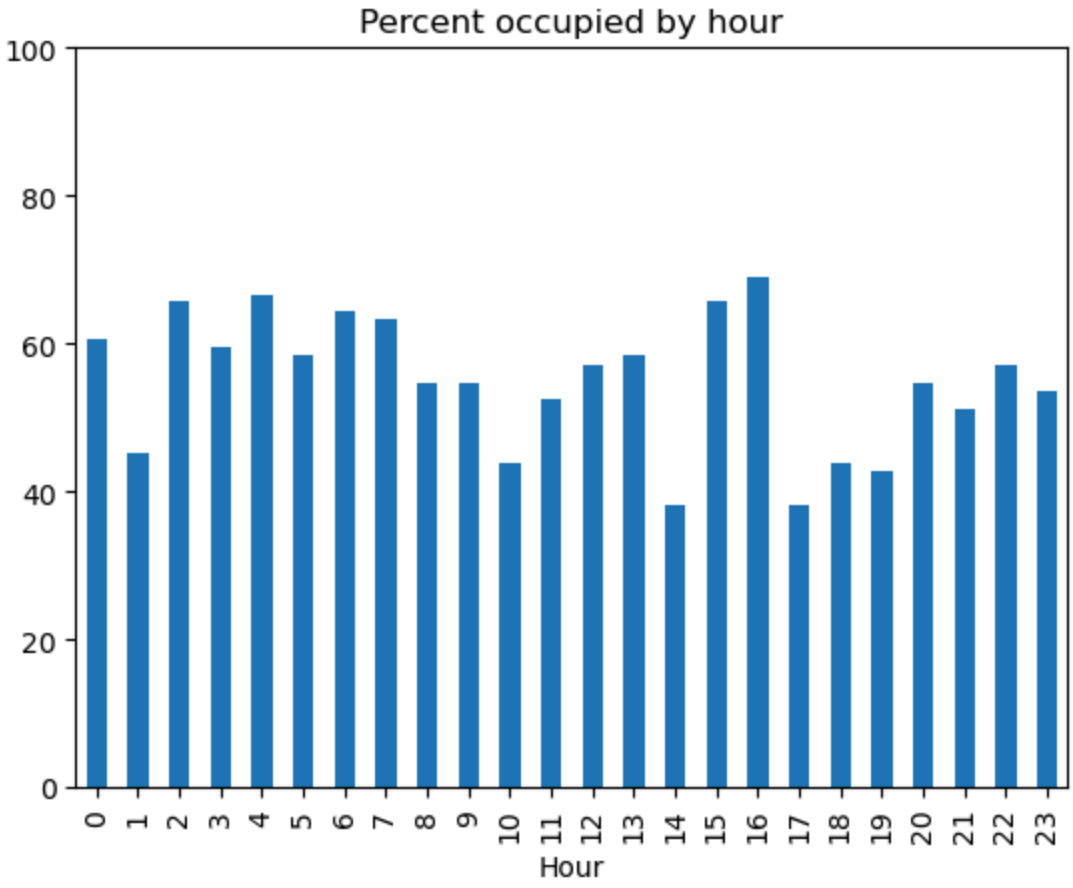
Out[5]:

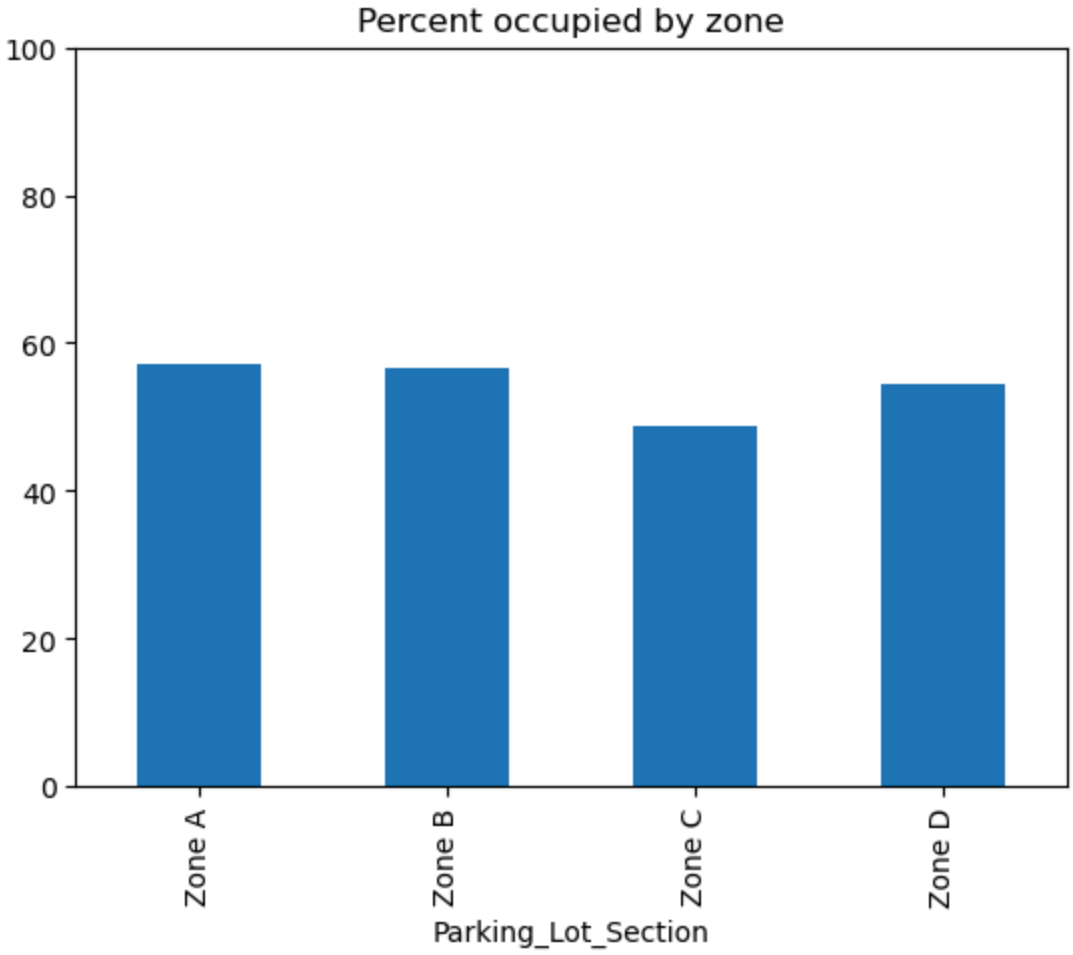
	Timestamp	Parking_Spot_ID	Parking_Lot_Section	Occupancy_Status	Hour	
0	2021-01-01 00:00:00.000000000	20	Zone A	Occupied	0	
1	2021-01-02 06:39:16.756756756	49	Zone A	Occupied	6	Sa
2	2021-01-03 13:18:33.513513513	38	Zone B	Vacant	13	S
3	2021-01-04 19:57:50.270270270	31	Zone B	Vacant	19	M
4	2021-01-06 02:37:07.027027027	8	Zone C	Occupied	2	Wedr

5. Look at the patterns

In [6]:
```
(df.groupby("Hour")["Occupied"].mean() * 100).plot(kind="bar", title="Percent occup
plt.ylim(0, 100)
plt.show()

(df.groupby("Parking_Lot_Section")["Occupied"].mean() * 100).plot(kind="bar", title
plt.ylim(0, 100)
plt.show()
```


Percent occupied by hour
100
80
60
40
20
0
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
Hour


Percent occupied by zone
100
80
60
40
20
0
Zone A
Zone B
Zone C
Zone D
Parking_Lot_Section

6. Split: old records to learn from, new records to test on

In [7]:
```
df = df.sort_values("Timestamp") #data sorted from oldest to newest, making predict

features = ["Hour", "Day", "Parking_Lot_Section"] #questions
target = "Occupied" #answers

X = df[features]
y = df[target]

cut = int(len(df) * 0.8) #of 1000 rows takes 80% converts to integer saved to cut (
X_train, y_train = X.iloc[:cut], y.iloc[:cut] #train 80
X_test, y_test = X.iloc[cut:], y.iloc[cut:] #test 20

print("learn rows:", len(X_train), " test rows:", len(X_test))
```

```
learn rows: 800  test rows: 200
```

7. The lazy guess (baseline)

In [8]:
```
from sklearn.dummy import DummyClassifier

lazy = DummyClassifier(strategy="most_frequent")
lazy.fit(X_train, y_train)
print("lazy guess score:", round(lazy.score(X_test, y_test), 3))
```

```
lazy guess score: 0.525
```

8. The real model: a decision tree

In [9]:
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, export_text

translator = ColumnTransformer(
    [("words", OneHotEncoder(handle_unknown="ignore"), ["Day", "Parking_Lot_Section
    remainder="passthrough",
)
tree = DecisionTreeClassifier(max_depth=3, random_state=42) # Max depth of 3 decisi
model = Pipeline([("translate", translator), ("tree", tree)]) #Translate data then

model.fit(X_train, y_train) #training the model, learn patterns and fitting to corr
print("tree score:", round(model.score(X_test, y_test), 3)) #print the score after
```

```
tree score: 0.565
```

In [10]:
```
names = model.named_steps["translate"].get_feature_names_out()
print(export_text(model.named_steps["tree"], feature_names=list(names)))
```

```
|--- remainder__Hour <= 5.50
|   |--- words__Parking_Lot_Section_Zone C <= 0.50
|   |   |--- words__Day_Saturday <= 0.50
|   |   |   |--- class: 1
|   |   |--- words__Day_Saturday >  0.50
|   |   |   |--- class: 1
|   |--- words__Parking_Lot_Section_Zone C >  0.50
|   |   |--- words__Day_Monday <= 0.50
|   |   |   |--- class: 1
|   |   |--- words__Day_Monday >  0.50
|   |   |   |--- class: 0
|--- remainder__Hour >  5.50
|   |--- words__Day_Friday <= 0.50
|   |   |--- remainder__Hour <= 16.50
|   |   |   |--- class: 1
|   |   |--- remainder__Hour >  16.50
|   |   |   |--- class: 0
|   |--- words__Day_Friday >  0.50
|   |   |--- words__Parking_Lot_Section_Zone C <= 0.50
|   |   |   |--- class: 1
|   |   |--- words__Parking_Lot_Section_Zone C >  0.50
|   |   |   |--- class: 0
```

This prints the questions the tree learned. Read it top to bottom like a flowchart.

- `Hour <= 5.50` means "is it before 6 am?"
- `Day_Friday <= 0.50` means "is it **not** Friday?"
- `class: 1` means it answers Occupied. `class: 0` means Vacant.

9. Mark the test properly (Detailed Breakdown of the results)

In [11]:
```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

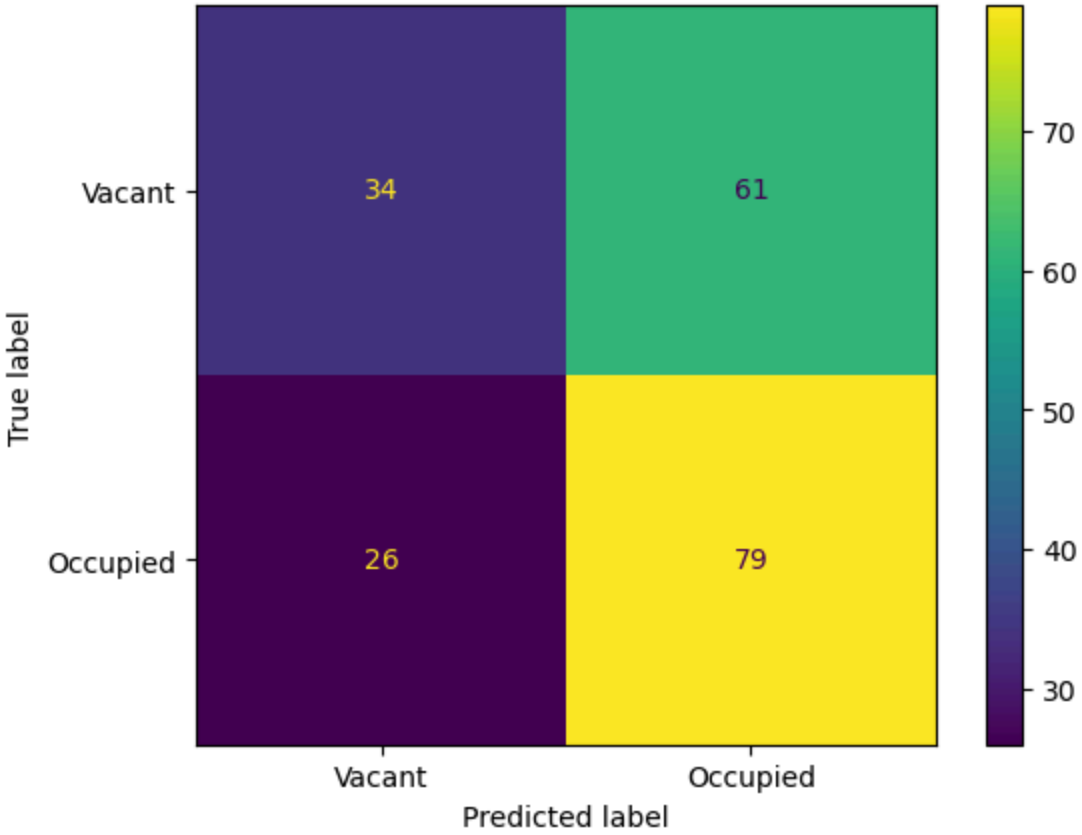
pred = model.predict(X_test) #predict for each row of 200
print(classification_report(y_test, pred, target_names=["Vacant", "Occupied"])) #ca

ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=["Vacant", "Oc
plt.show()
```

```
              precision    recall  f1-score   support

      Vacant       0.57      0.36      0.44        95
    Occupied       0.56      0.75      0.64       105

    accuracy                           0.56       200
   macro avg       0.57      0.56      0.54       200
weighted avg       0.57      0.56      0.55       200
```



10. Save the calculator, open it again, ask one question

In [12]:
```
import joblib

joblib.dump(model, "model/parkping_model.joblib")
loaded = joblib.load("model/parkping_model.joblib")

question = pd.DataFrame([{"Hour": 14, "Day": "Monday", "Parking_Lot_Section": "Zone
chance = loaded.predict_proba(question)[0][1]
print("Monday 14:00, Zone A:", round(chance * 100), "% chance occupied")
```

```
Monday 14:00, Zone A: 57 % chance occupied
```

11. Save the tidy table

In [13]:
```
df[["Timestamp", "Parking_Spot_ID", "Parking_Lot_Section", "Hour", "Day", "Occupied
    "data/final_parking_cleaned.csv", index=False
)
print("saved")
```

```
saved
```

The cleaned CSV contains the historical parking observations and can be used to display historical occupancy charts and statistics in ParkPing. The saved model is used to estimate occupancy for a selected day, hour and zone.